

운영을 염두에 두고 설계한 클라우드 인프라

Focus Tracking Platform — 웹캠 기반 학습 집중도 분석 풀스택 서비스.
대규모 트래픽 경험 대신 **관측성 · 자동화 · 배포 안정성 · 비용 최적화 · 보안**을 코
드(IaC)로 직접 구축하며 "운영이 굴러가는 구조"를 설계한 기록입니다.

CloudWatch Alarms → SNS

Datadog → Slack (Composite)

EventBridge / Auto Scaling 스케줄러

CodeDeploy Blue/Green

Firehose → S3 로그 파이프라인

GitHub OIDC · 최소권한 IAM

01 운영 한눈에 보기

"기능을 만든 것"을 넘어, 배포 후에도 스스로 감시·복구·절감되는 구조를 5개 축으로 코드화했습니다. 모든 항목은 Terraform으로 정의되어 재현·리뷰 가능합니다.

5축

관측성·자동화·배포·비용·보안

4종

CloudWatch 알람 → SNS

2채널

SNS(이메일)·Datadog(Slack)

~11h

야간 컴퓨팅 정지(22→09시)

관측성

CloudWatch 메트릭·알람, Datadog 통합 대시보드/트레이스, 로그 장기 보관 파이프라인까지 3계층.

자동화

CPU 기반 오토스케일링 + 야간 비용 절감 스케줄러(Fargate·EC2)를 시간/부하 트리거로 운영.

배포 안정성

CodeDeploy Blue/Green · 헬스체크 통과 후 전환 · 실패 시 자동 롤백.

비용 최적화

야간 스케줄러·ARM(Graviton)·단일 NAT·라이프사이클 등 트레이드오프를 의식한 결정.

보안

GitHub OIDC(정적키 없음)·Secrets Manager·서비스별 최소권한 IAM·전 구간 암호화.

IaC

전 인프라 Terraform 코드화, S3+DynamoDB 원격 상태, checkov 보안 스캔.

이 문서의 솔직한 전제 — 실 서비스의 대규모 트래픽을 받아본 경험은 없습니다. 대신 "트래픽이 왔을 때 자동으로 대응·감시·복구·절감되도록" 운영 구조를 먼저 설계했고, 각 수치(임계값·쿨다운·스케줄)를 왜 그렇게 정했는지 근거를 설명할 수 있는 것을 강점으로 삼습니다.

02 관측성 (Observability) — 3계층

"지표 → 알람 → 로그"를 분리해 구성했습니다. 즉각 통지가 필요한 신호와, 사후 분석을 위한 장기 보관을 분리합니다.

① CloudWatch Alarms → SNS(이메일)

알람	조건 (임계값)	의도
ALB 5xx 급증	5xx > 10 / 1분 (2회 평가)	백엔드 오류 폭증 감지
ECS Task 부족	RunningTaskCount < 1	서비스 다운 즉시 인지 (missing=breaching)
ECS CPU 과부하	평균 CPU > 80% / 3분	스케일아웃 한계·이상 부하
ALB 응답 지연	평균 응답 > 2초 / 3분	사용자 체감 성능 저하

② Datadog → Slack : Composite 모니터 (노이즈 억제 설계)

ML 서비스가 올라간 **t4g.small(버스터블)**의 함정 — CPU 크레딧이 소진되면 CPU%가 낮아 보여도 baseline으로 스로틀링되어 추론이 느려집니다. 단일 지표로는 못 잡습니다.

[입력 A] CPU 사용률 > 80% (5분) —AND—▶ [Composite] 스로틀링 위험
[입력 B] CPU 크레딧 잔량 < 50 (10분) —————▶ Slack #focus-alerts 통지

입력 모니터 2개는 **단독으로 알람을 보내지 않고**, 두 신호가 **동시(AND)**일 때만 Composite가 Slack을 울립니다. → "부하도 높고 크레딧도 바닥난 진짜 위험"만 통지해 **알람 피로(중복/노이즈)**를 억제한 설계입니다.

③ 로그 파이프라인 — 실시간 조회 + 장기 보관 분리

ECS 앱 로그 ▶ CloudWatch Logs ▶ Subscription Filter ▶ Kinesis Firehose ▶ S3 (GZIP · year/month/day 파티션)

- **버퍼링** 5MB 또는 300초 단위 flush, GZIP 압축으로 저장 비용 절감
- **ALB Access Logs** S3 직접 저장 · 라이프사이클(30일→Glacier IR, 90일 만료)
- **VPC Flow Logs** REJECT 트래픽만 수집 → 침해 탐지 핵심만 남기고 로그량 절감
- **Container Insights** ECS 컨테이너 단위 메트릭 활성화

※ Amazon Managed Grafana도 구성(grafana.tf)했으나, 도구를 늘리면 운영 복잡도·비용이 커져 Datadog로 일원화하고 비활성화 — "추가"가 아니라 "정리"한 운영 결정.

03 자동화 & 스케줄링

부하 기반(오토스케일링)과 시간 기반(스케줄러) 자동화를 함께 운용합니다.

① ECS Fargate 오토스케일링 — CPU Target Tracking

항목	값	근거
대상 지표	ECSServiceAverageCPUUtilization	Task 예약 CPU 대비 평균
목표값	75%	스케일 반영까지 2~3분 지연 → 그 사이 ~25% 버퍼 확보
범위	Task 1 → 2	최소 1(가용성), Fargate라 EC2 자리 제약 없음
scale-out 쿨다운	60초	부하 급증에 빠르게 대응
scale-in 쿨다운	300초	줄였다 늘리는 플래핑 방지

Fargate는 EC2 t4g 같은 CPU 크레딧/baseline 개념이 없어(요청한 vCPU를 그대로 받음) 목표값을 보수적으로 낮출 이유가 없습니다. → 플랫폼 특성에 맞춰 임계값을 선택한 사례.

② 야간 비용 절감 스케줄러 (dev) — 컴퓨팅을 0으로

낮에만 개발하므로 22:00 KST 정지 / 09:00 KST 기동. 시각은 코드 한 곳(local, KST)에서 관리.

대상	방식	동작
ECS Fargate	Auto Scaling Scheduled Action	밤 min/max=0(Task 0개) → 아침 min1/max2 복구
ML EC2	EventBridge Scheduler (AWS SDK universal target)	밤 StopInstances / 아침 StartInstances — Lambda 없이 API 직접 호출

- 컨테이너는 restart: unless-stopped → EC2 부팅 시 자동 복구되어 stop/start만으로 충분
- 전용 IAM Role에 해당 ML 인스턴스 ARN 하나만 Start/Stop 허용 → 최소 권한

효과 — 비업무 시간(약 11시간/일) ECS Task·ML EC2 컴퓨팅 비용 0. "쓰지 않는 자원은 끈다"를 코드로 자동화.

04 배포 안정성 & 보안

① CodeDeploy Blue/Green — 무중단 + 자동 롤백

신버전(Green) Task 기동 ▶ ALB 헬스체크 /api/health 통과 ▶ 트래픽 Blue→Green 전환 ▶ 5분 후 Blue 종료
실패 시(DEPLOYMENT_FAILURE) ▶ 자동 롤백 (트래픽 전환 안 함)

항목	값
트래픽 제어	WITH_TRAFFIC_CONTROL · BLUE_GREEN
자동 롤백	DEPLOYMENT_FAILURE 시
구버전 보존	성공 후 5분 대기 후 TERMINATE
타깃그룹	tg-blue / tg-green (헬스체크 /api/health)

② CI/CD — GitHub Actions, 정적 키 없음

- OIDC 장기 액세스 키 없이 IAM Role을 Assume — 키 유출 위험 제거
- 경로 트리거 backend/**→Blue/Green, ml-service/**→SSM Run Command, terraform/**→fmt·plan·apply
- ML 배포 SSH 미개방, GitHub Actions가 인스턴스를 태그로 찾아 SSM Run Command로 docker compose up

③ 보안 — 최소권한 · 시크릿 · 암호화

영역	적용
인증/시크릿	GitHub OIDC · RDS 비밀번호 Secrets Manager 관리(평문 없음)
IAM	서비스별 분리 Role(ecs-task / codedeploy / ml-ec2 / firehose / scheduler) — 최소 권한
네트워크 격리	앱·DB 프라이빗 서브넷, RDS 퍼블릭 차단, ML EC2 SSH 미개방(SSM), IMDSv2 강제
암호화	RDS·EBS(gp3)·S3(AES256)·ECR 저장 암호화, ALB TLS 1.3
IaC 스캔	checkov로 Terraform 보안 정적 분석

05 비용 최적화 & 트레이드오프 (정직한 한계)

"무조건 좋은 구성"이 아니라, dev 환경 제약 안에서 **비용과 가용성을 의식적으로 맞바꾼 결정들**입니다. 면접에서 트레이드오프를 함께 설명할 수 있는 것이 핵심.

결정	효과	맞바꾼 것 (한계)
야간 스케줄러(dev)	비업무 시간 컴퓨팅 0	야간 무중단 불가 — dev라 허용
단일 NAT Gateway	NAT 시간/처리 비용 절감	AZ 장애 시 아웃바운드 단절 (이상적으론 AZ별 2개)
ARM64 / Graviton	동급 x86 대비 가격·전력 효율	일부 x86 전용 바이너리 비호환 가능성
FE+BE 단일 컨테이너	Task 수·관리 포인트 축소	프론트/백 독립 스케일 불가
RDS PostgreSQL (vs Aurora)	초기 비용↓	트래픽 증가 시 Aurora Serverless v2 이전 여지 남김
Grafana 비활성화	모니터링 Datadog 일원화	오픈소스 대시보드 자체 운영 포기

06 예상 면접 질문 & 답변 포인트

Q. 트래픽 경험이 없는데 오토스케일링이 의미가 있나요?

대규모 트래픽은 못 받아봤지만, **CPU 75% 목표값과 쿨다운(out 60s / in 300s)을 직접 정했고 그 이유를 설명**할 수 있습니다. 75%는 스케일 반영 지연(2~3분)을 버틸 버퍼, 비대칭 쿨다운은 플래핑 방지 — 수치의 근거를 말할 수 있다는 게 강점입니다.

Q. 왜 알람을 지표 하나가 아니라 Composite로 묶었나요?

t4g 버스터블 특성 때문입니다. CPU 크레딧이 소진되면 **CPU%는 낮는데 실제로는 스로틀링**되는 사각지대가 생깁니다. 그래서 'CPU 높음 AND 크레딧 낮음'을 AND로 묶어 **진짜 위험만 Slack** 통지하고 노이즈를 줄였습니다.

Q. CloudWatch와 Datadog을 둘 다, Grafana는 왜 켜나요?

CloudWatch는 AWS 네이티브 알람/로그(저비용), Datadog는 통합 대시보드·트레이스·CSPM 담당으로 **역할을 분담**했습니다. Grafana도 구성해봤지만 도구가 늘면 운영 복잡도·비용이 커져 **일원화**했습니다 — 운영은 더하는 것뿐 아니라 **줄이는 결정**도 포함 된다고 봅니다.

Q. 배포 중 장애는 어떻게 막나요?

Blue/Green으로 신버전이 **헬스체크(/api/health)를 통과한 뒤에만** 트래픽을 전환하고, 실패하면 자동 롤백, 구버전은 5분간 보존 합니다. 사용자 입장에서 무중단입니다.

Q. 비용은 어떻게 관리했나요?

dev 야간 스케줄러로 22~09시 컴퓨팅 0, ARM/Graviton, 단일 NAT, ECR(최근 5개)·S3 라이프사이클을 적용했습니다. 각각이 무엇을 절감하고 무엇을 희생하는지 트레이드오프까지 함께 설명할 수 있습니다.

한 줄 정리 — "기능을 배포하는 것"에서 멈추지 않고, 배포 이후에도 감시되고(관측성) · 스스로 대응하며(자동화) · 안전하게 바뀌고(배포) · 새는 비용을 막는(최적화) 운영 구조를 IaC로 직접 설계했습니다. 트래픽 규모 경험은 앞으로 채우되, 운영을 시스템으로 사고하는 관점은 이미 갖췄습니다.